Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Operator-blocked	1
Bug	Queued	15
Task	In progress	3
Task	Operator-blocked	3
Task	Queued	33
TOTAL		**55**

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-hos SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jackett + compose env-forward
BOB-068	Bug	Queued	High	RD2-00: unattributed, unreviewed Auto commit mechanism pushing to main
BOB-069	Task	Queued	High	RD2-40: §11.4.238 QA-discovery-channe ledger — ongoing coverage-escape back
BOB-070	Bug	Queued	High	RD2-41: pre-build mutation-marker scan carrier false-positive silently defeats entire pre-build gate
BOB-074	Task	Queued	Medium	RD2-07: DDoS-class testing fully absent from the mandated test-type matrix
BOB-077	Task	Operator-blocked	High	RD2-10: Identify second host running th Auto-commit rsync/sync mechanism (OPERATOR-DECISION)

ATM ID	Type	Status	Severity	Description
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire
BOB-079	Task	Queued	Medium	RD2-12: Retroactive attributed history notes for GA-18/21/22/25/26/27 changes (never rewrite published history)
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit diffs
BOB-081	Task	Queued	High	RD2-14: Author CONTINUATION.md Session 15 entry (currently 53 days / 24 commits behind HEAD)
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)
BOB-083	Task	Queued	Medium	RD2-16: Regenerate browser_extension features/codegraph Status.md + Summary/HTML/PDF siblings
BOB-084	Task	Queued	High	RD2-17: Reconcile BOB-008 DB/MD boo drift via the workable-items tool
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/ merge-service v1.0.0 readiness ledger (closes GA-10)
BOB-086	Task	Queued	Medium	RD2-19: Fix BOB-009/BOB-010 evidence_path + backfill item_history fo 56 silent closures
BOB-087	Task	Queued	High	RD2-20: Wire docs_chain / commit-seam sync hook per §11.4.106(F) so DB write cannot land without MD mirror
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tracked-Items + Status Documents tabl row-completeness (GA-07 remainder)
BOB-089	Task	Queued	High	RD2-24: RED-first tests for start.sh reload_python/reload_plugins/ recreate_stack (closes test-half of GA-27
BOB-090	Task	Queued	Medium	RD2-25: HelixQA Challenge entry exercising all three start.sh subcommand end-to-end against real compose stack
BOB-092	Bug	Queued	Medium	RD2-27: Remove test_live_stack_evidence.py:265 nnmclu SKIP-on-404 fallback + verify live 200 (closes GA-13)
BOB-093	Task	Queued	High	RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-

ATM ID	Type	Status	Severity	Description
BOB-094	Task	Queued	Medium	RD2-29: Author tests/stress/ test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection
BOB-095	Task	Queued	Medium	RD2-30: Author tests/stress/ test_scheduler_hooks_sse_stress_chaos. for Go-side triangle
BOB-096	Task	Queued	Medium	RD2-31: Extend qBitTorrent-go jackett_db_test.go with real process-kill resource-exhaustion fault injection
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage fo exposed download-proxy/merge endpoi (canonical impl of RD2-07)
BOB-098	Task	Queued	Medium	RD2-34: Parametrize 20 hardcoded / Volumes/T7 paths in helixqa banks with PROJECT_ROOT (closes GA-23)
BOB-099	Bug	Queued	Medium	RD2-36: Fix guard-forbidden-commands.sh substring-match false-positive class + add const033-poweroff-signal-triage carrier to EXCLUDE_PATH
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)
BOB-101	Task	Operator-blocked	High	GA-19/RW-09: Is –profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	Operator-blocked	Medium	RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0? – OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-105	Task	Queued	Medium	§11.4.238 followup: mechanical §11.4.227(B) anchor-block-integrity che
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescenc check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existe check for subagent task-brief source inputs
BOB-109	Task	Queued	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-ty matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test covera (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba’s 3 public HTTP endpoints

ATM ID	Type	Status	Severity	Description
BOB-113	Task	Queued	Low	BOB-074 followup: add wrk to dev tooli for DDoS/load challenges
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation golde bad fixture for the rate-limit detector
BOB-118	Bug	Queued	High	README.md python-tests badge claims 585 passing; pytest –collect-only measu 5235 (9x stale/wrong)
BOB-120	Bug	Queued	Critical	3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap
BOB-121	Task	Queued	Important	External watchdog for the forced-logou architectural gap (task #85, incident #3
BOB-129	Bug	Queued	Medium	Potential production slowapi/starlette defect flagged by Task 105 subagent
BOB-131	Bug	Queued	Medium	qbittorrent-proxy podman conmon crash — pre-existing, surfaced during BOB-12 investigation
BOB-135	Bug	Queued	Low	Test isolation: test_list_hooks_after_crea fails in bulk suite (Permission denied / config)
BOB-136	Task	Queued	High	WHAT. A single 2026-08-20 sweep found FOUR workable-item rows whose tracke state contradicted reality: BOB-076 (closed by commit 99a486e on 2026-08-15, whose message literally say “closes BOB-076”, still Queued), BOB-1 (fix landed AND covered by a permanen guard assertion, still Queued), BOB-091 (implemented under d1a8479 / a3b16bc 2a0b543, still Queued), and BOB-008 whose Evidence field still quoted a diagnostic string that BOB-117 removed from source. CORRECTION 2026-08-20 (11.4.6). An earlier revision of this item claimed the enabling defect was that “workable-items diff is blind to body_md drift”. That was WRONG and is correcte here rather than quietly edited. The engine is NOT blind. The FLAGLESS INVOCATION is: workable-items diff –db docs/workable_items.db -> “diff: DB and Markdown are in sync” (opens ZERO .md files) workable-items diff –db docs/workable_items.db –issues docs/Issues.m –fixed docs/Fixed.md -> “~ BOB-008 bo differs (md=1346 bytes db=1333 bytes) -> “diff: 1 difference(s)” Both measured

ATM ID	Type	Status	Severity	Description
				the same seeded body-only drift. An independent agent proved the mechanis with strace: the flagless form never ope any Markdown file. So this is a 11.4.201(6) FALSE-NULL of the CALLE not a gap in the comparison logic. Importantly, pre_build_verification.sh invariant 17 already passes both flags, s the STANDING GATE was never blind – the drift above was not caused by a blin gate. THE REAL DEFECTS, restated honestly. (1) The flagless form is a false null trap: it reports a confident, reassur "in sync" while checking nothing. Any caller using it is a green-reporting blind check. It should either default to the conventional Issues.md/Fixed.md paths REFUSE with a clear error, never retur clean verdict for a comparison it did not perform. (2) Work commonly lands unde commit messages that do not carry the ATM id (GA-14/15/16 for BOB-091), so nothing mechanically links a commit ba to its row, and closure depends on a human remembering. 11.4.227: an anch done state is its SEAM landing, not its TEXT landing. ACCEPTANCE. (a) The flagless diff invocation no longer return clean verdict without reading Markdow with a paired 1.1 mutation proving it. (b A repo-wide grep confirms no caller use the flagless form. (c) A sweep that flags any non-terminal item whose id appears a merged commit message, so done-but open rows are found mechanically rathe than by a human noticing. (d) Honest boundary: this does not claim to preven all drift. RELATED. scripts/hooks/docs-sync-commit-seam.sh (added under BOB-087) now enforces doc/DB sync at the commit seam and passes the flags explicitly, plus an independent body_md re-parse oracle. EVIDENCE. docs/qa/ BOB-117/closure-evidence.md, docs/qa/ BOB-076/closure-evidence.md, docs/qa/ BOB-091/closure-evidence.md.
BOB-137	Bug	Queued	High	**Reported-Via:** §11.4.202 reporting directive bug on 2026-08-20T14:46:52Z **Reported-By:** Claude **What (the repo verbatim):** The merge service on port

ATM ID	Type	Status	Severity	Description
				7187 wedges after hours of uptime: the port stays bound and the container keep reporting “healthy”, but every HTTP request hangs until the client times out Measured 2026-08-20 16:41 UTC+2 on container up 3h53m: curl http://localhost:7186/ -> HTTP 200 in 0.096s (proxy, same process) curl http://localhost:7187/ -> HTTP 000 after 6.0s (merge service, WEDGED) Both ports a served by the SAME process (pid 2330069, fd 3 = 7186, fd 7 = 7187), so this is not a crashed worker – one loop inside a live process has stopped servic requests while another in the same process is fine. Thread state at the time the wedge (/proc/2330069/task, 16 threads): - tid 2330529: state R, wchan <- ONE thread spinning on CPU - tid 2330528: state S, wchan do_sys_poll <- the healthy 7186 poll loop - the other 14 state S, wchan futex_wait_queue That is the GIL-starvation signature: a thread busy-looping in Python holds the GIL a every other thread queues on the GIL futex. 7186 survives because do_sys_po releases the GIL; the 7187 async loop needs sustained GIL time to service a request and starves. Process CPU was 20.6% while idle. Corroborating evidenc seven sockets held by the process sit in CLOSE-WAIT with unread bytes still in t receive queue (Recv-Q 162, 162, 162, 7 1, 1, 1) – clients sent a request and hun up, and the app never read it or closed fd. The listener had also accumulated a unaccepted backlog (Recv-Q 6 on the 7 LISTEN socket) at first observation. The container log’s last line is 2h before the observation, so the service processed nothing in that window. NOT fd exhaustion: only 48 of 16384 fds were open. ROOT CAUSE NOT ESTABLISHE All four `while True` loops in the service (routes.py:166, search.py:1169, streaming.py:161, streaming.py:359) a correctly bounded and awaited, so the spin is not a naive unslept loop. py-spy could not attach to name the spinning frame – the host has

ATM ID	Type	Status	Severity	Description
				kernel.yama.ptrace_scope=1, which denies non-child attach. Per the §11.4.1 Iron Law no fix is proposed until the spinning frame is identified. Next diagnostic step: obtain a Python stack. Either run py-spy INSIDE the container (musl wheel), or set ptrace_scope=0 for the duration of one dump, or add a SIGQUIT/faulthandler.register() dump hook to main.py so the running service can be asked for its own stacks without ptrace. DISCOVERY CHANNEL (§11.4.238): found by an agent probing host by hand while investigating an unrelated test-suite result – NOT by automated QA. This is a coverage escap in its own right; see the sibling item on the health check that was structurally incapable of observing it. **Affected scope / file-scope manifest:** downloac proxy/src/main.py, download-proxy/src/ merge_service/, download-proxy/src/api streaming.py, docker-compose.yml (qbittorrent-proxy) **Reproduction / context:** Leave the qbittorrent-proxy container running for several hours wit search traffic (a full tests/security run is sufficient). Then: curl –max-time 6 http: localhost:7186/ returns 200; curl –max- time 6 http://localhost:7187/ returns 00 Confirm with: ss -ltnp | grep 7187 (unaccepted backlog) and awk '{print $ /proc/<pid>/task/*/stat (one R thread, r futex_wait_queue). **Acceptance criteria:** The spinning frame is identif from a real Python stack dump (not inferred), the busy-loop is fixed at its ro and a regression guard proves 7187 stil answers after a sustained-traffic soak. Evidence: before/after curl timings on both ports plus a thread-state census showing no permanently-R thread. | | BOB-141 | Task | Queued | Low | **Reported-Via:** §11.4.202 reporting directive `task` on 2026-08-20T14:56:3 **Reported-By:** Claude **What (the report, verbatim):** CLAUDE.md's Architecture section states: "qbittorrent proxy-go (Go/Gin, opt-in via --profile go) replaces the Python proxy on 7186, 718

ATM ID	Type	Status	Severity	Description
				7188" The container cannot deliver that Read from source rather than prose: qBitTorrent-go/Dockerfile:16 CMD ["/ap qbittorrent-proxy"] (ONE binary) qBitTorrent-go/Dockerfile:15 EXPOSE 7187 7188 (declares 2) cmd/qbittorrent proxy/main.go r.Run(fmt.Sprintf(":%d", cfg.ServerPort)) (binds 1) internal/confi config.go:58 ServerPort = MERGE_SERVICE_PORT (default 7187) the qbittorrent-proxy-go container binds 7187 ONLY. webui-bridge is a SEPARAT binary (cmd/webui-bridge, /bridge/healt cfg.BridgePort) that this container neve starts, and nothing in it binds 7186 eith -- although the compose service does se PROXY_PORT=7186 and BRIDGE_PORT=7188 in its environmen which reinforces the wrong impression. EXPOSE likewise declares a port nothin binds. WHY THIS MATTERS BEYOND TIDINESS: this prose was used as the source of truth when first authoring config/served_ports.yaml, producing a manifest entry of [7186, 7187, 7188] for that service. The healthcheck gate built it then FAILED a service whose healthcheck was already correct -- a §11.4.201(1) false-positive refusal cause directly by trusting the doc over the Dockerfile. The manifest was corrected against source; the doc was not, and wi mislead the next reader the same way. Either the doc is wrong, or the Go container is under-provisioned relative intent (it should also run webui-bridge a a 7186 listener). Determining WHICH is part of this task -- do not simply reword the doc to match the current binary if th intended design was a three-port container. Related: the Go service's compose block sets PROXY_PORT and BRIDGE_PORT that no process in the container consumes, and EXPOSE lists 7188 unbound. Whichever way the abov resolves, those should agree with reality afterwards. **Affected scope / file-scope manifest:** CLAUDE.md (Architecture - Port Map), docker-compose.yml (qbittorrent-proxy-go env/EXPOSE),

ATM ID	Type	Status	Severity	Description
				qBitTorrent-go/Dockerfile **Reproduction / context:** Read qBitTorrent-go/Dockerfile:16 (single CM against CLAUDE.md's 'replaces the Python proxy on 7186, 7187, 7188'; confirm cfg.ServerPort resolves to MERGE_SERVICE_PORT=7187 in internal/config/config.go:58. **Acceptan criteria:** Doc and container agree, wit the direction of the fix decided deliberately (correct the doc, or provisio the container to match the documented intent). PROXY_PORT/BRIDGE_PORT en and EXPOSE lines agree with what the container actually binds. | | BOB-143 | E | Queued | Medium | **Reported-Via:** §11.4.202 reporting directive `bug` on 2026-08-20T15:08:18Z **Reported-By:** Claude **What (the report, verbatim):** `.worktrees/ci-split-workflows/` (13M) a `.worktrees/completion-initiative-phase- ` (33M) are ORPHANED: `git worktree list` reports only the main checkout, so neither is a registered worktree. Each contains a `.git` POINTER FILE whose target gitdir no longer exists, so git can resolve HEAD, branch, or status inside them -- every query returns empty. They are gitignored (.gitignore:122), so nothi tracks them and nothing will ever notice them drifting. WHY THIS IS NOT COSMETIC: they pollute the scan scope whole-tree gates and manufacture false findings. Measured 2026-08-20 by the §11.4.32 sweep: - `cm_test_mock_pid_explicit_int` reporte 2 violations at tests/unit/merge_service/ test_deadline_tunable.py:44 -- BOTH inside these orphaned trees. The MAIN tree's copy of that file is already harden (it sets `mock.pid = 12345` and patches os.killpg/os.getpgid) and passes the gat cleanly. The finding read as a live §11.4.263 / BOB-126-class defect and w not one. - 6 of the 57 "missing anchor carrier" files flagged by the propagation gates were likewise `.worktrees/**`. Th is the §11.4.201(1) false-positive shape sourced from scan scope, and it costs re investigation time: a reader triaging "2

ATM ID	Type	Status	Severity	Description
				live BOB-126 violations" reasonably trea it as a host-safety emergency. CLARIFICATION (established during triage, so the record is not alarming): these trees are NOT a kill(-1) vector. Th `download-proxy/src/merge_service/ search.py` contains ZERO `os.killpg` ca -- they predate that cleanup code entire -- so there is no unguarded signal call to reach. Additionally `pyproject.toml` set `testpaths = ["tests"]`, so a plain `pytes run does not collect from `.worktrees/`. No host-safety risk was found. The defe is scan-scope noise plus 46M of unreferenced disk. WHY REMOVAL IS NOT DONE AUTONOMOUSLY (§11.4.122 / §11.4.124 / §11.4.101): because git cannot resolve their HEAD, is NOT possible to prove their contents are merged into main. Deleting unprovable-provenance work is exactly the irreversible, operator-owned decisio §11.4.122 reserves. Two options for the operator: (a) confirm removal (they are stale dev scratch dirs) -- reversible only from backup, so a §9.2 pre-op backup should precede it; or (b) keep them and add `.worktrees/` to the gate scan-scop exclusion list as a §11.4.224(E)-fenced, checked-in, justified entry. Either way th exclusion list is the cheaper immediate mitigation and does not destroy anythin **Affected scope / file-scope manifest:** .worktrees/ci-split-workflows/, .worktrees/completion-initiative-phase-0/, gate scan-scope conf **Reproduction / context:** git worktree list shows only the main checkout; git -C .worktrees/<dir> log -1 returns empt (gitdir target missing). Run the §11.4.32 sweep and observe cm_test_mock_pid_explicit_int report 2 violations, both under .worktrees/, while the main-tree file passes the same gate. **Acceptance criteria:** Whole-tree gat no longer report findings sourced from orphaned worktrees: either the dirs are removed after operator confirmation wi a §9.2 pre-op backup, or .worktrees/ is added to a checked-in §11.4.224(E)-fenc

ATM ID	Type	Status	Severity	Description
				exclusion list with justification. Verify by re-running the sweep and confirming zero .worktrees-sourced findings. | | BOB-144 | Bug | Queued | Low | **Reported-Via:** §11.4.202 reporting directive `bug` on 2026-08-20T15:53:57 **Reported-By:** Claude **What (the report, verbatim):** `/theme/stream` in download-proxy/src/api/routes.py:167 c the disconnect probe with NO guard at while True: if await request.is_disconnected(): break If that probe raises, the generator dies with an UNCAUGHT exception. IMPORTANT — this is NOT the BOB-139 fail-open. The effect here is fail-CLOSED: the stream stops, and the enclosing `finally: store.unsubscribe(queue)` still runs, so the subscriber queue is released and nothing leaks. The outcome is CORREC the manner is not. What is wrong with i it terminates via an uncaught traceback rather than a clean SSE `close` event, s the client sees a truncated stream inste of a reason; - the failure is not logged as probe failure, so a systematically raising probe would show up as recurring tracebacks with no diagnosis; - it is inconsistent with the two SSE generato in streaming.py, which after BOB-139 e `event: close` with reason `disconnect_probe_failed` and log a warning. Three call sites of the same probe now behave two different ways. T BOB-139 fix deliberately did not touch t file (ownership boundary, §11.4.119), an flagged it honestly rather than fixing it of scope. Acceptance: /theme/stream us the same probe-failure discipline as the streaming.py generators — a clean clos event with the `disconnect_probe_failed reason plus a logged warning — proven BOTH directions (§11.4.201(1)): a raisin probe closes the stream cleanly AND a normally-connected client still streams uninterrupted. Prefer reusing the share helper introduced by BOB-139 rather th a third copy of the logic (§11.4.251). Honest boundary (§11.4.6): the product probe-failure rate is UNKNOWN —

ATM ID	Type	Status	Severity	Description
				nobody has measured how often `is_disconnected()` actually raises. This filed on the inconsistency and the missi diagnosis, not on a measured incident rate. **Affected scope / file-scope manifest:** download-proxy/src/api/ routes.py (~line 167, stream_theme) **Reproduction / context:** Monkeypat Request.is_disconnected to raise, open theme/stream, observe the generator di with an uncaught traceback rather than emitting event: close with reason disconnect_probe_failed as streaming.p now does. **Acceptance criteria:** Sam probe-failure discipline as streaming.py (clean close + disconnect_probe_failed reason + logged warning), verified both directions, reusing the BOB-139 helper rather than a third copy. | | BOB-145 | B | Queued | High | **Reported-Via:** §11.4.202 reporting directive `bug` on 2026-08-20T16:01:43Z **Reported-By:** Claude **What (the report, verbatim):** BOB-137 established the root cause: Deduplicator.merge_results() is called a PLAIN SYNCHRONOUS CALL at merge_service/search.py:914 and therefore executes ON the asyncio even loop thread. While it runs, uvicorn's onl loop runs no callback — no accept, no read, no write — so port 7187 stops answering entirely. Measured episodes 9m37s and 5m56s, self-clearing, recurri under ordinary traffic. This item is the FIX. BOB-137 is the diagnosis and stays separate per the §11.4.102 Iron Law — investigation deliberately applied no fix Three candidate directions, not yet chosen: (a) OFFLOAD — hand merge_results to a thread executor (asyncio.to_thread / run_in_executor). Smallest change, immediately unblocks the loop. Does NOT make the work cheaper, so a large enough merge still burns a core; and it introduces concurrency around self._last_merged_results and metadata which must be checked for races before is called safe. (b) MEMOISE — _normalize_name is called at FOUR site

ATM ID	Type	Status	Severity	Description
				per comparison, each running 5 re.sub, and _extract_identity_from_result twice more with a ~15-re.search chain. lru_cache has ZERO matches in that module today, so the seed's own normalisation is recomputed for every candidate. Caching the per-result normalisation is a large constant-factor win with no concurrency risk. (c) REDU THE COMPARISON SET — the O(N^2) shape itself (blocking/bucketing by a cheap key before pairwise comparison). Largest win, largest change, highest ris of altering dedup RESULTS — which would need its own correctness evidenc not just a speed measurement. (b) then is the likely order: (b) is risk-free and m alone bring the merge under the threshold, and (a) guarantees the loop i never blocked regardless. MANDATORY for whoever takes this: - RED FIRST (§11.4.43/§11.4.224): a test that FAILS o the current code by demonstrating the loop is blocked during a merge — e.g. assert a concurrent request to 7187 is served within a bounded time while a large merge runs. A pure speed benchmark is NOT the RED test; the defect is loop-blocking, not slowness. - BOTH POLARITIES (§11.4.201(1)): the loop stays responsive under a large mer AND dedup results are unchanged for t existing corpus. A fix that speeds up merging while changing which duplicat are detected is a different defect. - Use existing instrument: scripts/diagnostics bob137_soak.sh reproduced the wedge 22/26; it is the natural GREEN check. - Honest boundary carried from BOB-137 measured growth is SUPER-LINEAR bu not fully quadratic at N<=400 (2.4-3.4x per doubling vs 4.0). Worst case is quadratic BY CODE STRUCTURE. Do n cite "measured N^2". **Affected scope file-scope manifest:** download-proxy/s merge_service/search.py:914, downloa proxy/src/merge_service/deduplicator.p **Reproduction / context:** bash scripts diagnostics/bob137_soak.sh — reproduc the wedge in 22/26 probes (7187 dead,

ATM ID	Type	Status	Severity	Description
				7186 alive throughout). **Acceptance criteria:** Port 7187 answers within a bounded time while a large merge runs (loop never blocked), AND dedup result are unchanged for the existing corpus. Proven with the soak repro flipping fro 22/26-dead to 0/N-dead, plus a dedup-equivalence check. | | BOB-146 | Bug | Queued | High | **Reported-Via:** §11.4.202 reporting directive `bug` on 2026-08-20T16:10:50Z **Reported-By:* Claude **What (the report, verbatim):** The constitution's §11.4.252 detector (constitution/scripts/gates/ cm_dangerous_combination_fail_closed. UNDERCOUNTS by 29%. Measured against an independent AST instrument (structure, not text — a valid control needle per §11.4.201(7)) on boba's plug tree: gate reports : 30 AST ground truth 42 missed : 12 false positives : 0 A gate that misses 12 of 42 while reporting a confident number is a §11.4.201(6) false null: its output reads as a census when is a FLOOR. Anyone fencing an exclusio list against that number (§11.4.224(E)) would write the fence against an undercount and lock the invisible sites o of scope permanently. FOUR DISTINCT CAUSES, each independently falsifiable L1 — TRAILING COMMENT DEFEATS THE REGEX (10 of the 12). The pattern anchors the handler line with `...: [[:space:]]*$, soexcept Exception: # no S110never matches. The irony is load-bearing: the sites a human consciously reviewed and annotated are exactly the ones the gate cannot see. L2 — TUPLE CLAUSE DEFEATS THE REGEX (2 of the 12). The exception-type group is[A-Za-z_.]+, which cannot match(, soexcept (OSError, ValueError):is invisible. L3 — A COMME BETWEENexceptANDpassDEFEATS THE BODY CHECK. The detector reads exactlylineno+1. Zero current instances but demonstrated live. This one has a nasty second-order effect: a well-intentioned reviewer adding an explanatory comment INSIDE a handler mak that site vanish from the gate. The

ATM ID	Type	Status	Severity	Description
				triage agent hit this itself — its first patch put comments inside two handlers and the resulting count would have read while only 6 sites were genuinely eliminated. It caught and corrected that rather than reporting the better number which is exactly the §11.4 discipline working. L4 — THE SHAPE ITSELF, not the regex. The body must be exactlypass, soexcept: return